Backend Introduction

A Spring Boot backend application for a RAG (Retrieval-Augmented Generation) based intelligent Q&A system. This system enables users to upload documents and engage in intelligent conversations based on the document content using OpenAI's GPT models and vector embeddings.

System demo: http://downey.top/login

Table of Contents

Technology Stack

Core Framework

- **Java 17** - Programming language
- **Spring Boot 3.3.3** - Application framework
- **Spring Security** - Authentication and authorization
- **MyBatis 3.0.4** - ORM framework (SSM architecture)

Database

- **MySQL 8.4.0** - Relational database
- **Flyway** - Database migration tool

AI & ML

- **LangChain4j 0.33.0** - Java framework for LLM applications
- **OpenAI API** - GPT-4o-mini for chat, text-embedding-3-small for embeddings
- **InMemoryEmbeddingStore** - Vector storage for document embeddings

Security

- **JWT (jjwt 0.11.5)** - Token-based authentication

Document Processing

- **Apache PDFBox** - PDF document parsing
- **Apache POI 5.2.5 & 4.1.2** - Word document parsing (.doc, .docx)

Build Tools

- **Maven** - Dependency management and build tool

System Architecture & Features

Architecture Overview

The system follows a layered architecture:

```
Controller Layer (REST API)
    ↓
Service Layer (Business Logic)
    ↓
Mapper Layer (Data Access)
    ↓
Database (MySQL)
```

Core Features

1. User Authentication & Authorization

- User registration with role-based access (Admin/User)
- JWT token-based authentication
- Password encryption using BCrypt
- Token expiration and refresh handling

2. Document Management

- **Multi-format Support**: PDF, TXT, MD, HTML, DOC, DOCX
- **Document Parsing**: Automatic text extraction from various formats
- **Vector Indexing**: Documents are split into chunks and converted to embeddings
- **Document Storage**: File upload and metadata management
- **Index Rebuilding**: Automatic re-indexing on application startup

3. RAG (Retrieval-Augmented Generation) System

- **Semantic Search**: Vector similarity search using OpenAI embeddings
- **Context Retrieval**: Top-K retrieval (K=4) of relevant document chunks
- **Intelligent Q&A**: GPT-4o-mini generates answers based on retrieved context
- **Reference Tracking**: Tracks and displays source documents for each answer
- **Context Awareness**: Maintains conversation history (last 10 messages)

4. Chat System

- **Multi-session Management**: Users can create multiple chat sessions
- **Message History**: Persistent storage of chat messages
- **Streaming Response**: Server-Sent Events (SSE) for real-time token streaming
- **Non-streaming Support**: Traditional request-response mode

API Endpoints

Authentication

- `POST /api/auth/register` - User registration
- `POST /api/auth/login` - User login

Document Management (Admin only)

- `POST /api/document/upload` - Upload document
- `GET /api/document/list` - List all documents
- `GET /api/document/{id}/preview` - Preview document
- `DELETE /api/document/{id}` - Delete document

Chat

- `POST /api/chat/create` - Create new chat session
- `GET /api/chat/list` - List user's chat sessions
- `GET /api/chat/{chatId}/history` - Get chat history
- `POST /api/chat/{chatId}/send` - Send message (non-streaming)
- `POST /api/chat/{chatId}/stream` - Send message (streaming, SSE)

Local Development

Prerequisites

- Java 17 or higher
- Maven 3.6+
- MySQL 8.0+
- OpenAI API Key

Setup Steps

1. **Clone the repository**

   ```
   git clone <repository-url>
   cd capstone
   ```

2. **Configure MySQL Database**

   Create a MySQL database:

   ```
   CREATE DATABASE rag_chat CHARACTER SET utf8mb4 COLLATE
   utf8mb4_unicode_ci;
   ```

   Update database credentials in `src/main/resources/application.yml`:

   ```
   spring:
     datasource:
       url: jdbc:mysql://localhost:3306/rag_chat?
   useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Shanghai&
       username: root
       password: your_password
   ```

3. **Set Environment Variables**

   Set your OpenAI API key:

   ```
   export OPENAI_API_KEY=sk-your-api-key-here
   ```

   Or configure it in `application.yml`:

```
app:
  openai:
    api-key: sk-your-api-key-here
```

4. **Configure File Directories**

   Update paths in `application.yml`:

   ```
   app:
     rag:
       index-dir: /path/to/vector-index
       upload-dir: /path/to/uploads
   ```

5. **Build the Project**

   ```
   mvn clean install
   ```

6. **Run the Application**

   ```
   mvn spring-boot:run
   ```

   Or run the JAR file:

   ```
   java -jar target/rag-chat-backend-0.0.1-SNAPSHOT.jar
   ```

7. **Verify the Application**

   The application will start on `http://localhost:8080`

   Check if the API is running:

   ```
   curl http://localhost:8080/api/auth/login
   ```

Database Migration

Flyway will automatically run database migrations on application startup. Migration scripts are located in `src/main/resources/db/migration/`.

Development Notes

- The application uses Flyway for database schema management
- Document indexing happens automatically on upload
- Vector index is rebuilt on application startup via `RagBootstrap`
- Logs are configured in `logback-spring.xml` and output to `logs/app.log`

Docker Deployment

For Docker deployment instructions, please refer to DEPLOYMENT_GUIDE.md.

The deployment guide includes:

- Docker and Docker Compose setup
- Environment configuration
- Service orchestration
- Data persistence
- Logging and monitoring
- Troubleshooting

Quick Docker Deployment

1. Create `.env` file with required environment variables
2. Build and start services:

```
docker compose up -d --build
```

3. Check service status:

```
docker compose ps
```

For detailed deployment steps, see DEPLOYMENT_GUIDE.md.

Frontend Introduction

A modern React-based frontend application for the RAG Chat intelligent Q&A system. This single-page application provides a user-friendly interface for document management and AI-powered conversations based on uploaded documents.

System demo: http://downey.top/login

Table of Contents

Technology Stack

Core Framework

- **React 19.1.1** - UI library
- **TypeScript 5.8.3** - Type-safe JavaScript
- **Vite 7.1.2** - Build tool and dev server

Routing & Navigation

- **React Router DOM 7.8.2** - Client-side routing

Styling

- **Tailwind CSS 3.4.17** - Utility-first CSS framework
- **@tailwindcss/typography 0.5.19** - Typography plugin for markdown content
- **PostCSS 8.5.6** - CSS processing
- **Autoprefixer 10.4.21** - CSS vendor prefixing

HTTP Client

- **Axios 1.12.0** - Promise-based HTTP client with interceptors

State Management

- **Jotai 2.14.0** - Atomic state management
- **React Hooks** - Built-in state management

Internationalization

- **i18next 25.6.0** - Internationalization framework
- **react-i18next 16.2.1** - React bindings for i18next
- **Supported Languages**: English, Chinese (Simplified)

UI Components & Utilities

- **react-markdown 10.1.0** - Markdown rendering for AI responses
- **Sonner 2.0.7** - Toast notification library
- **class-variance-authority 0.7.1** - Component variant management
- **clsx 2.1.1** - Utility for constructing className strings

Development Tools

- **ESLint 9.33.0** - Code linting
- **TypeScript ESLint 8.39.1** - TypeScript-specific linting rules
- **@vitejs/plugin-react 5.0.0** - Vite plugin for React

System Architecture & Features

Architecture Overview

The frontend follows a component-based architecture with clear separation of concerns:

```
Pages (Route Components)
    ↓
API Layer (Axios with interceptors)
    ↓
Backend API (REST + SSE)
```

Core Features

1. User Authentication

- **Login Page**: User authentication with JWT token storage
- **Register Page**: User registration with role selection (Admin/User)

- **Token Management**: Automatic token injection in API requests
- **Auto-redirect**: Unauthorized users are redirected to login
- **Session Persistence**: Token stored in localStorage

2. Chat Interface

- **Multi-session Management**: Create and manage multiple chat conversations
- **Real-time Streaming**: Server-Sent Events (SSE) for token-by-token response streaming
- **Message History**: Display conversation history with user and AI messages
- **Markdown Rendering**: Rich text formatting for AI responses
- **Auto-scroll**: Automatic scrolling to latest message
- **Optimistic Updates**: Immediate UI updates for better UX
- **Reference Display**: Show source documents for AI answers

3. Document Management (Admin Only)

- **Document Upload**: Multi-format file upload (PDF, TXT, MD, HTML, DOC, DOCX)
- **Document List**: Display all uploaded documents with metadata
- **Document Preview**: View document content in modal
- **Document Deletion**: Remove documents from knowledge base
- **Role-based Access**: Only admin users can access document management

4. Internationalization (i18n)

- **Language Switching**: Toggle between English and Chinese
- **Persistent Language**: Language preference saved in localStorage
- **Complete Translation**: All UI text translated for both languages

5. User Experience

- **Responsive Design**: Mobile and desktop optimized
- **Modern UI**: Clean, gradient-based design with Tailwind CSS
- **Toast Notifications**: User feedback for actions (success/error)
- **Loading States**: Visual feedback during async operations
- **Error Handling**: Graceful error handling with user-friendly messages

Page Structure

- `/login` - User login page
- `/register` - User registration page
- `/chat` - Main chat interface (default route)
- `/docs` - Document management page (admin only)

API Integration

The frontend communicates with the backend through:

- **REST API**: Standard HTTP requests for CRUD operations
- **SSE (Server-Sent Events)**: Streaming chat responses
- **Axios Interceptors**: Automatic token injection and error handling

Key API modules:

- `AuthAPI` : Authentication endpoints
- `ChatAPI` : Chat and conversation management

- `DocsAPI` : Document upload and management

Local Development

Prerequisites

- **Node.js** 18+ (recommended: Node.js 20+)
- **npm** or **yarn** package manager
- Backend API running on `http://localhost:8080`

Setup Steps

1. **Navigate to frontend directory**

   ```
   cd frontend
   ```

2. **Install dependencies**

   ```
   npm install
   ```

   Or using yarn:

   ```
   yarn install
   ```

3. **Start development server**

   ```
   npm run dev
   ```

   The application will start on `http://localhost:5173` (default Vite port)

4. **Access the application**

   Open your browser and navigate to:

   ```
   http://localhost:5173
   ```

Development Scripts

- **`npm run dev`** - Start development server with hot module replacement (HMR)
- **`npm run build`** - Build production bundle to `dist/` directory
- **`npm run preview`** - Preview production build locally
- **`npm run lint`** - Run ESLint to check code quality

Configuration

API Proxy Configuration

The development server is configured to proxy API requests to the backend. This is configured in `vite.config.ts` :

```
server: {
  proxy: {
    '/api': {
      target: 'http://localhost:8080',
      changeOrigin: true,
    },
  },
}
```

If your backend runs on a different port, update the `target` in `vite.config.ts`.

Environment Variables

For production builds, you may need to configure the API base URL. Create a `.env` file:

```
VITE_API_BASE_URL=http://localhost:8080
```

Then update `src/lib/api.ts` to use the environment variable if needed.

Development Notes

- **Hot Module Replacement (HMR)**: Changes are reflected instantly without full page reload
- **TypeScript**: Full type safety with TypeScript
- **ESLint**: Code quality checks on save (if configured in your IDE)
- **Browser DevTools**: React DevTools recommended for debugging

Building for Production

1. **Build the application**

   ```
   npm run build
   ```

   This creates an optimized production build in the `dist/` directory.

2. **Preview the production build**

   ```
   npm run preview
   ```

3. **Deploy the `dist/` directory**

   The `dist/` directory contains all static files needed for deployment. You can:

   - Serve with any static file server (Nginx, Apache, etc.)
   - Deploy to CDN or cloud storage
   - Use with Docker (see Docker Deployment section)

Docker Deployment

For Docker deployment instructions, please refer to DEPLOYMENT_GUIDE.md.

The deployment guide includes:

- Docker and Docker Compose setup
- Frontend build process in Docker
- Nginx configuration for serving static files
- API proxy configuration
- Production deployment steps

Quick Docker Deployment Overview

1. **Build the frontend**

   ```
   npm run build
   ```

2. **Copy `dist/` contents** to the deployment directory
3. **Configure Nginx** to serve static files and proxy API requests
4. **Start services** with Docker Compose

For detailed deployment steps, see DEPLOYMENT_GUIDE.md.

Nginx Configuration

The project includes an `nginx.conf` file for production deployment. Key features:

- Serves static files from `/usr/share/nginx/html`
- Proxies `/api/*` requests to backend service
- Single-page application routing support (fallback to `index.html`)
- File upload size limit: 50MB